

Architecting Multi-Agent Swarms: Orchestration, Cost Optimization, and Quality Assurance

Parvez Syed Mohamed

Abstract

The transition from single-agent coding assistants to collaborative Multi-Agent Systems (MAS) introduces significant execution capabilities but exposes engineering teams to novel failure modes, context window bloat, and escalating API costs. This paper presents a field-tested Orchestrator-Worker-Adversary (OWA) architecture designed to mitigate these inefficiencies. Validated through empirical builds and supported by recent research on multi-agent collaboration and tokenomics, we demonstrate that specialized role isolation, local-first execution, and strict context pruning significantly reduce latency (up to 50%) and token costs (up to 65%) while ensuring high-quality code generation. This synopsis and the accompanying findings provide a highly actionable blueprint for deploying cost-effective, scalable agentic frameworks suitable for production-grade software engineering and arXiv submission.

1. Introduction & Strategic Scoping

The transition from isolated Large Language Models (LLMs) to collaborative Multi-Agent Systems (MASs) represents a paradigm shift in autonomous software engineering. However, a common failure mode in agentic development is allowing an LLM to generate an exhaustive, unedited task list. A 200-task execution plan is not an asset; it is a liability.

Before engaging the swarm, top-tier reasoning models must be utilized strictly for Product Requirements Document (PRD) decomposition and scope reduction. Force the model to aggressively challenge the execution plan, define a rigid P0 Minimum Viable Product, and deprecate non-essential features. The primary ROI of expensive, high-parameter models in the planning phase is identifying what *not* to build.

2. The Orchestrator-Worker-Adversary (OWA)

Architecture

Recent surveys of LLM collaboration mechanisms emphasize that centralized, role-based topologies are crucial for managing complex interactions. Building on this, the OWA architecture assigns agents opposing incentives. Code quality scales positively when planning, generation, and destructive testing are separated into isolated roles. A central orchestration layer is non-negotiable to manage handoffs, state, and tool execution without descending into copy-paste chaos.

2.1 Model Allocation Strategy

Role	Tier / Cost	Primary Function	Operational Directive
Orchestrator	Flagship (~\$0.50-3.00/response)	Judgment, Dispatch, Merging	Maintains global state. Requires high reasoning, low hallucination, massive context retention.
Worker	Mid-Tier (Per-token generation)	Execution, Generation	Optimized for polite, accurate code generation and mechanical tool execution.
Adversary	Subscription (\$0 per-call)	Destructive Testing	Incentivized strictly to break the Worker's code. Writes aggressive, failing tests.
Verifier	Flagship (~\$0.50-3.00/block)	Integration Assurance	Runs <i>once per block</i> to catch cross-subtask integration issues. Do not run per-task.

Note: Model capability is role-specific, not monolithic. Trialing free or cheaper models for the Orchestrator role often results in missed adversary breaks, wrong merges, and poor task

scoping.

2.2 The Adversary Role and Separation of Powers

Role isolation must extend beyond prompts to the tool layer. The Orchestrator must not edit code; its profile must be stripped of file-patching tools entirely. Every fix must route through a Worker subagent and pass through the Adversary. The necessity of an independent testing agent is corroborated by recent work demonstrating that execution-grounded verification and role decomposition independently drive system performance and correctness beyond what single agents can achieve.

3. Infrastructure and Cost Optimization

Agentic workflows generate hidden taxes through network latency, API round-trips, and context window bloat. The following steps outline how to eliminate these taxes.

Step 1: Shift to Local-First Execution

Routing every agent subtask through hosted CI introduces unacceptable latency and burns unnecessary compute minutes. Furthermore, execution feedback inside a mandatory, local sandbox provides a much stronger supervision signal than simulated next-token predictions.

Prescription: Run all gates, linters, and tests on the host machine. Push to remote repositories only at the conclusion of a milestone block.

Step 2: Context and Schema Pruning

Recent tokenomic analyses of agentic software engineering reveal that the refinement and code review stages account for nearly 60% of total token consumption, with input tokens constituting the vast majority of this cost. Loading an agent's system prompt with comprehensive Model Context Protocol (MCP) schemas creates massive bloat paid for on every iterative API call.

Prescription: Enforce strict tool pruning. Disable irrelevant MCP servers before dispatching backend tasks.

Step 3: Parallel Gate Execution

Sequential gate runs (build → test → lint) consume unnecessary minutes. **Prescription:** Run verification components as parallel background sessions. This collapses the wait time to the latency of the slowest single component.

Step 4: Path-Filtered CI

When using self-hosted runners, every push triggers all gates. **Prescription:** Attach path filters to each block gate so it only runs when its own code or shared dependencies change, skipping slow suite-wide tests for localized package changes.

4. Measured Impact

The optimizations described above were validated across a multi-block build run. Agent attention and cloud round-trips are the two largest controllable line items in an agentic pipeline. By shifting to a local-first workflow and aggressively pruning context, teams can expect dramatic reductions in overhead.

Metric	What It Means	Before Optimization	After Optimization	Net Savings
Wall clock per subtask	Time from task start to merged and verified.	40-80 min	20-45 min	~50% reduction
Token cost per block	Dollars spent on model API calls per milestone block.	\$15-40	\$5-15	~65% reduction
Cloud CI Minutes	Billed compute time on hosted runners.	Rapid quota exhaustion	0	100% (Local-first)
System prompt overhead	Irrelevant schemas shipped per API call.	~20KB per call	0	~20KB saved per call
Subtask Context Footprint	Tokens orchestrator consumes per subtask processing.	20-30K tokens	~5KB tokens	~99% reduction
Verifier invocations	Paid API calls to the integration Verifier agent.	Per-subtask	Once per block	~89% reduction

5. Operational Pitfalls and Safeguards

Deploying autonomous agents reveals systemic edge cases that are often misdiagnosed as model failures:

- **No-Fallback Hard Stop:** If an assigned model is unavailable, halt the pipeline. Do not silently substitute a cheaper model for the Orchestrator, as this reintroduces judgment failures. Fail loudly.
- **Transport Timeouts:** Ensure orchestration layer timeout configurations align with the maximum expected execution time of the agent to prevent silent drops.
- **Directory Isolation Breaches:** Never name the main repository path in the context window; naming it is an implicit instruction to navigate there and corrupt the main branch.
- **Stale State Validations:** Force cache invalidation at the start of every verification gate. A passing lint result relying on a cached state from a previous run is worse than no gate at all.
- **Asynchronous Racing:** Require explicit, synchronous blocks before initiating corrective branches to prevent fix tasks and feature tasks from racing.

6. Post-Build Audit

An agent's closure comment ("CI green, all tests pass") is a claim, not evidence. Agents frequently struggle with long-horizon trajectories and diagnosing their own errors in complex repository-level tasks. Relying purely on an agent's self-report is insufficient. Before declaring a block complete, run an independent audit against the remote repository state:

1. Confirm fix commits are present on the remote, not just local.
2. Confirm CI is actually green on the remote main branch.
3. Re-run end-to-end tests locally to confirm they pass on the final merged code.
4. Re-validate any security suppressions for factual accuracy.

The OWA loop produces a durable audit trail only if a human or independent audit framework verifies it against ground truth after the loop closes.

7. Conclusion: When OWA Is Overkill

The OWA pattern is justified when a subtask touches security, state machine transitions, or anything where a regression is expensive. It is *not* justified for mechanical changes (e.g., a one-line config bump or doc typo). Running a full loop on a trivial change burns feature-level costs for zero additional assurance. Maintain a fast path for trivial work.

The integration of LLMs into Multi-Agent Systems marks a significant shift toward autonomous problem-solving. However, a swarm is best treated as an engineering organization to be managed. The cost and quality wins come from assigning separated roles, opposing incentives, and strict tool isolation, rather than treating the system as a single omniscient model to be prompted.

References

- He, J., Treude, C., & Lo, D. (2025). LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *ACM Transactions on Software Engineering and Methodology*.
- Kumar, R. (2026). AgentForge: Execution-Grounded Multi-Agent LLM Framework for Autonomous Software Engineering. *arXiv preprint*.
- Salim, M. (2025). Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering. *arXiv preprint*.
- Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O'Sullivan, B., & Nguyen, H. D. (2025). Multi-Agent Collaboration Mechanisms: A Survey of LLMs. *arXiv preprint*.
- Wang, M., Xie, X., & Huo, Y. (2026). TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. *arXiv preprint*.